

数字电路设计

第 2 章——计算机组成概述

考试导向学习笔记

考试重要性：【重点】

主讲教师：任胜兵 中南大学计算机学院软件工程系

整理日期：2026 年 6 月

考试重点提示：

计算机性能分析——主频与计算机工作速度的关系，

主频越大 → 计算机越快是**必要条件**（非充分条件）。

重点掌握：CPI、MIPS、执行时间、MTBF、Amdahl 定律与加速比计算。

目 录

1 计算（机）的发展	3
1.1 世界计算发展四阶段	3
1.2 中国计算机发展四代	3
2 存储程序概念与冯·诺依曼结构	4
2.1 存储程序概念（1946 年 6 月，冯·诺依曼等提出）	4
2.2 计算机系统组成	4
2.3 冯·诺依曼结构 vs 哈佛结构	4
3 ARM 体系结构	5
3.1 ARM 架构演进	5
3.2 片上系统（SoC）	5
3.3 ARM 公司发展简史	5
4 计算机的工作过程	6
4.1 指令执行五步骤	6
5 重点 性能及主要技术指标（考试重点）	7
5.1 关键结论：主频与计算机速度的关系	7
5.2 字长和主频	7
5.3 存储容量	7
5.4 【重点】 运算速度核心指标	7
5.4.1 CPI —— 每条指令执行所需时钟周期数	7
5.4.2 IPC —— 每个时钟周期执行的指令数	7
5.4.3 MIPS —— 每秒百万条指令数	8
5.4.4 CPU 执行时间	8
5.4.5 完整计算示例（教材 P27 例题）	8
5.4.6 MFLOPS —— 每秒百万次浮点操作	9
5.5 可靠性、可用性和可维修性	9
5.6 兼容性和性价比	9
6 【重点】 Amdahl 定律与加速比计算（考试重点补充）	11
6.1 Amdahl 定律核心思想	11
6.2 基本公式	11
6.2.1 加速比定义	11
6.2.2 公式推导	11
6.3 Amdahl 定律的三个重要推论	12
6.4 经典计算示例	12
6.4.1 示例 1：基本加速比计算	12
6.4.2 示例 2：极限加速比	12

6.4.3 示例 3: 多部件并行优化	13
6.5 Amdahl 定律的现实意义	13
7 性能评价方法与工具	15
7.1 CPU 性能评价方法	15
7.2 常用基准程序	15
7.3 性能测试工具	15
8 【重点】 考试要点速查表	16
8.1 核心公式汇总	16

1 计算（机）的发展

1.1 世界计算发展四阶段

时代	时期	代表成就
手工时代	远古-公元前	石头、手指、绳子计数；商周时代算筹
机械时代	公元前 5 世纪-公元 17 世纪	算盘（中国，公元前 5 世纪）；对数计算尺（欧洲，17 世纪）
电子时代	1946 年-至今	ENIAC （1946 年 2 月 15 日）：第一台电子数字计算机，重 30 多吨，每秒 5000 次加法运算
量子时代	2020 年代-	“九章三号” （2023 年，中国）：光量子计算机原型，可操纵 255 个光子，打破世界纪录

1.2 中国计算机发展四代

1. 第一代：电子管计算机（1958-1964）

- 1958 年 8 月 1 日：我国第一台电子数字计算机 **103 型（DJS-1 型）** 诞生（仿制苏联）
- 1964 年：第一台自行设计的大型通用电子管计算机 **119 机** 研制成功

2. 第二代：晶体管计算机（1965-1972）

- 1964 年 11 月：第一台晶体管通用电子计算机 **441-B 机** 研制成功

3. 第三代：中小规模集成电路计算机（1973-80 年代初）

- 1973 年：运算速度每秒 100 万次的大型通用计算机

4. 第四代：超大规模集成电路计算机

- 1983 年：**DJS-0520 微机**（与 IBM PC 兼容）
- 1983 年：国防科技大学**银河-I 巨型机**（每秒上亿次运算）

2 存储程序概念与冯·诺依曼结构

2.1 存储程序概念（1946 年 6 月，冯·诺依曼等提出）

三大核心要点：

1. 计算机硬件由五大基本部件组成：运算器、控制器、存储器、输入设备、输出设备
2. 计算机内部采用二进制表示指令和数据
3. 将编好的程序和原始数据事先存入存储器中，再启动计算机工作——存储程序的基本含义

2.2 计算机系统组成

计算机系统		
硬件	主机	中央处理器（CPU）：运算器 + 控制器
		内存储器
	外部设备	外存储器
		输入设备
		输出设备
软件	系统软件	操作系统、语言处理程序、服务性程序
	应用软件	通用软件、用户程序

2.3 冯·诺依曼结构 vs 哈佛结构

对比维度	冯·诺依曼结构	哈佛结构
存储器	指令和数据共享同一存储器	两个独立存储器模块，分别存储指令和数据
总线	一条总线，指令和数据分时复用	两条独立总线，分别连接 CPU 与各存储器
存储方式	指令和数据按执行顺序依次存储	指令和数据不允许并存于同一模块
典型代表	ARM7（ARM7TDMI）	ARM9（ARM968）
特点	结构简单，成本低	并行访问，效率更高

注意：第一台电子计算机 ENIAC 不是冯·诺依曼计算机——它的内存很小，程序和数据存储方式不同。存储程序概念由冯·诺依曼等人在 1946 年 6 月才首次提出。

3 ARM 体系结构

3.1 ARM 架构演进

架构版本	发布年份	主要特性
ARM V9	2021 年	安全性提升、AI 性能增强、整体性能提升。推出 Cortex X2、Cortex A710、Cortex A510。X2 和 A510 仅支持 AArch64（64 位），A710 兼容 32 位
ARM V10	2023 年	SVE2 向量处理、FP16 半精度浮点原生支持、MTE2 增强型内存标记扩展、SVE2 Crypto 可扩展向量加密指令。截至 2025 年末尚无量产芯片标注“ARMv10”认证

3.2 片上系统（SoC）

SoC（System on a Chip）可包含：

- **CPU**：中央处理单元
- **FPGA**：可编程单元
- **DSP**：数字信号处理器
- **On-Chip Peripherals**：片上外设

3.3 ARM 公司发展简史

- **1978 年**：Hermann Hauser 和 Chris Curry 在剑桥创办 CPU 公司
- **1979 年**：改名 Acorn 计算机公司
- **1985 年**：Roger Wilson 和 Steve Furber 设计第一代 32 位 6MHz 处理器——**ARM (Acorn RISC Machine)**
- **1990 年**：改组为 ARM 计算机公司（苹果出资 150 万英镑，VLSI 出资 25 万英镑）
- **2006 年**：全球 ARM 芯片出货量 20 亿片
- **2010 年**：出货量达 60 亿片
- **2016 年**：软银以 310 亿美元收购 ARM
- **当前市场**：移动计算（>85%）、企业基础架构（15%）、嵌入智能（25%）；正发力服务器、汽车和物联网领域

4 计算机的工作过程

实质：周而复始地取出指令、解释指令和执行指令的过程。

4.1 指令执行五步骤

1. **取指令 (Fetch)**：将程序计数器 PC 中的指令地址送存储器的地址寄存器 AR，按该地址取出指令送指令寄存器 IR
2. **取操作数**：根据 IR 中的地址码，由地址计算部件形成操作数地址送存储器，取出数据送到运算器中的寄存器（寄存器组）
3. **译码 (Decode)**：将 IR 中的操作码 OP 送指令译码器进行译码
4. **执行 (Execute)**：计算机有关部件在控制器发出的操作控制信号的控制下，执行操作码 OP 规定的操作
5. **更新 PC**：程序计数器 PC 加 1，形成下一条指令地址

指令周期流程：

取指令 → 取操作数 → 译码 → 执行 → PC+1 → （循环）

5 重点 性能及主要技术指标（考试重点）

5.1 关键结论：主频与计算机速度的关系

考试必考结论：

主频越大 → 计算机越快，这是**必要条件**，但不是充分条件。

原因：计算机的运算速度还受到 CPI（每条指令所需时钟周期数）、指令系统、存储器速度、I/O 性能、编译器优化等多种因素的影响。即使主频很高，如果 CPI 也很高（指令执行效率低），实际运算速度仍然可能较慢。

5.2 字长和主频

- **字长**：计算机中每次运算的数所包含的二进制数的位数，是 8 的倍数。字长标志着机器表示数的精度——位数越多，精度越高。
- **主频率 (f)**：计算机的主时钟不断产生固定频率的时钟脉冲，时钟脉冲的频率就是 CPU 的主频率。CPU 工作的节拍由主频率控制，主频率越高，CPU 工作节拍越快——是影响计算机运行速度的**重要参数**。

5.3 存储容量

- **主存容量（往往较小）**：CPU 可直接访问的存储器。例：具有 16 位地址码的计算机，主存容量为 $2^{16} = 64K$ 字节。
- **外存容量（往往较大）**：联机外存储器的容量，存放暂不参与运行的程序和数据。

5.4 【重点】运算速度核心指标

5.4.1 CPI —— 每条指令执行所需时钟周期数

$$CPI = \frac{\text{程序执行所需的总时钟周期数}}{\text{程序包含的总指令条数}}$$

CPI 的重要性：CPI 越小，说明每条指令平均需要的时钟周期越少，CPU 执行效率越高。主频相同的情况下，CPI 更小的 CPU 运算速度更快——这正是主频不是速度充分条件的原因。

5.4.2 IPC —— 每个时钟周期执行的指令数

$$IPC = \frac{1}{CPI}$$

5.4.3 MIPS ——每秒百万条指令数

$$\text{MIPS} = \frac{\text{指令条数 (条)}}{\text{执行时间 (s)} \times 10^6} = \frac{f \times \text{IPC}}{10^6} = \frac{f}{\text{CPI} \times 10^6}$$

参数说明：

- f : CPU 主频率 (Hz)
- CPI: 每条指令平均所需时钟周期数
- IPC: 每个时钟周期执行的指令数 ($= 1/\text{CPI}$)

5.4.4 CPU 执行时间

$$T = \frac{\text{指令数} \times \text{CPI}}{f}$$

5.4.5 完整计算示例 (教材 P27 例题)

已知条件：

某程序包含 2×10^5 条指令，CPU 主频 $f = 10^9 \text{ Hz} = 1 \text{ GHz}$ 。
四种类型指令的 CPI 和出现频率如下：

指令类型	CPI	出现频率
类型 A	1	60%
类型 B	2	18%
类型 C	4	12%
类型 D	8	10%

求解：

1. 平均 CPI
2. MIPS 值
3. 程序执行时间 T

解：

Step 1 ——计算平均 CPI:

$$\begin{aligned}
 \text{CPI}_{\text{avg}} &= 1 \times 60\% + 2 \times 18\% + 4 \times 12\% + 8 \times 10\% \\
 &= 0.60 + 0.36 + 0.48 + 0.80 \\
 &= \boxed{2.24}
 \end{aligned}$$

Step 2 ——计算 MIPS:

$$\begin{aligned}
 \text{MIPS} &= \frac{f}{\text{CPI} \times 10^6} = \frac{10^9}{2.24 \times 10^6} \\
 &= \frac{10^9}{2.24 \times 10^6} = \frac{1000}{2.24} \\
 &= \boxed{446.4 \text{ (每秒 446.4 百万条指令)}}
 \end{aligned}$$

Step 3 ——计算执行时间:

$$\begin{aligned}
 T &= \frac{\text{指令数} \times \text{CPI}}{f} = \frac{2 \times 10^5 \times 2.24}{10^9} \\
 &= \frac{4.48 \times 10^5}{10^9} = 4.48 \times 10^{-4} \text{ s} \\
 &= \boxed{0.448 \text{ ms}}
 \end{aligned}$$

5.4.6 MFLOPS ——每秒百万次浮点操作

$$\text{MFLOPS} = \frac{\text{浮点操作次数}}{\text{执行时间 (s)} \times 10^6}$$

参考网站: <http://www.top500.org> (全球超级计算机排名)

5.5 可靠性、可用性和可维修性

指标	定义	公式
可靠性 (MTBF)	故障平均间隔时间	$\text{MTBF} = \frac{\text{系统有效使用时间}}{\text{故障次数}}$
可用性	系统可正常使用的时间占比	$\text{可用性} = \frac{\text{系统有效使用时间}}{\text{系统有效使用时间} + \text{故障修复时间}}$
可维修性	故障修复的平均时间	平均修复时间越短, 可维修性越好

5.6 兼容性和性价比

- **向上兼容 (向前兼容):** 在较低档计算机上编写的程序, 可以在同一系列的较高档计算机上运行 (例: Intel 386 的软件可在更高档机型运行)

- **向下兼容（向后兼容）**：用旧版本程序创建的文档或系统仍能被新版本正常操作（例：Word 2007 能打开 Word 2003 的文件）
- **性价比**：性能与价格的比例关系，受处理器、内存、外部设备等硬件组件的性能和价格影响

6 【重点】Amdahl 定律与加速比计算（考试重点补充）

6.1 Amdahl 定律核心思想

Amdahl 定律 (Amdahl's Law) 是计算机系统设计中最重要性能分析定律之一, 由 Gene Amdahl 于 1967 年提出。该定律揭示了系统中对某一部件进行改进所能获得的总体性能提升的极限。

核心思想:

对系统中某一部件采用某种更快执行方式后, 整个系统性能的提高, 受限于该部件在系统中被使用的频率 (占比)。

也就是说: 优化用得多的部分才有效果, 用得少的部分优化再多也收效甚微。

6.2 基本公式

6.2.1 加速比定义

$$S = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{1}{(1-f) + \frac{f}{n}}$$

参数说明:

- S : 系统加速比 (Speedup Ratio), $S > 1$ 表示有性能提升
- f : 被改进部件在原系统中占用的时间比例 ($0 \leq f \leq 1$), 即可改进比例
- $(1-f)$: 不可改进部分占用的时间比例
- n : 被改进部件的性能提升倍数 ($n > 1$), 即部件加速比

6.2.2 公式推导

设原系统执行时间为 $T_{\text{old}} = 1$ (归一化):

- 可改进部分原时间 = f
- 不可改进部分时间 = $1-f$
- 改进后可改进部分时间 = f/n
- 改进后总时间 $T_{\text{new}} = (1-f) + f/n$

因此加速比:

$$S = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{1}{(1-f) + f/n}$$

6.3 Amdahl 定律的三个重要推论

1. **推论一（改进上限）**：当 $n \rightarrow \infty$ 时（即某部件被无限加速，时间趋近于 0），系统的极限加速比为：

$$S_{\max} = \frac{1}{1-f}$$

这意味着，即使某个部件快到不花时间，系统的整体加速比也不超过 $\frac{1}{1-f}$ ，受限于不可改进部分。

2. **推论二（优化优先级）**：要使系统获得显著的加速，必须大幅提升在系统中占比较大的部分的性能。对于占比较小的部件（ f 接近 0），即使极大地加速该部件（ n 很大），对系统的整体性能提升也非常有限。
3. **推论三（成本效益）**：改进 f 较小但 n 很大的部件，不如改进 f 较大但 n 适中的部件。优化应聚焦于瓶颈部件。

6.4 经典计算示例

6.4.1 示例 1：基本加速比计算

题目：

某计算机系统执行一个任务时，CPU 处理时间占 70%（ $f = 0.7$ ），I/O 等待时间占 30%。现对 CPU 进行升级，使其速度提高 4 倍（ $n = 4$ ）。问：整个系统的加速比是多少？

解：

$$\begin{aligned} f &= 0.7, \quad 1-f = 0.3, \quad n = 4 \\ S &= \frac{1}{(1-f) + f/n} = \frac{1}{0.3 + 0.7/4} = \frac{1}{0.3 + 0.175} = \frac{1}{0.475} \\ &= \boxed{2.105} \end{aligned}$$

结论： CPU 速度提升 4 倍，整个系统只快了约 2.1 倍。因为 I/O 等待时间占 30% 保持不变，限制了整体加速。

6.4.2 示例 2：极限加速比

题目：

若某系统中某类运算占总执行时间的 80%。如果对该运算部件进行无限加速（ $n \rightarrow \infty$ ），系统最高能获得多少倍的加速？

解：

$$f = 0.8$$

$$S_{\max} = \frac{1}{1-f} = \frac{1}{1-0.8} = \frac{1}{0.2} = \boxed{5}$$

结论：即使把这 80% 的运算加速到不花时间，系统最多也只能快 5 倍。还有 20% 的不可加速部分始终限制着性能。

6.4.3 示例 3：多部件并行优化

题目：

某程序执行过程中，浮点运算占 50%，整数运算占 30%，访存占 20%。若将浮点部件加速 10 倍，整数部件加速 2 倍，访存部件不变。求系统整体加速比。

解：使用广义 Amdahl 公式（多个部件同时改进）：

$$S = \frac{1}{(1-f_1-f_2) + \frac{f_1}{n_1} + \frac{f_2}{n_2}}$$

$$f_1 = 0.5, n_1 = 10; \quad f_2 = 0.3, n_2 = 2; \quad 1 - f_1 - f_2 = 0.2$$

$$S = \frac{1}{0.2 + \frac{0.5}{10} + \frac{0.3}{2}} = \frac{1}{0.2 + 0.05 + 0.15} = \frac{1}{0.40} = \boxed{2.5}$$

注意：虽然浮点部件加速了 10 倍，但由于访存部件（20%）未改进，系统整体加速比只有 2.5——这就是 Amdahl 定律揭示的瓶颈效应。

6.5 Amdahl 定律的现实意义

- **多核处理器的性能限制：**当程序中有串行部分（不可并行化）时，即使有无限多个处理器核心，加速比也受限于 $\frac{1}{\text{串行比例}}$ 。这也是为什么“主频大战”之后，多核并未带来线性加速。
- **系统设计指导：**找到系统的性能瓶颈（占用时间最多的部分），优先优化它。
- **与主频关系的联系：**提高主频只加速了 CPU 执行速度（ f 的一部分），如果存储系统、I/O 等不随之改进，总体加速比会受到 Amdahl 定律的限制——这就是主频越高不等于计算机越快的定量解释。

主频 $\uparrow \neq$ 速度 $\uparrow$ 的定量原因（串联考点）：

设某计算机：CPU 执行占 60% ($f = 0.6$)，存储访问占 40%。若主频提高 2 倍（CPU 加速 $n = 2$ ，假设理想情况），则系统加速：

$$S = \frac{1}{0.4 + 0.6/2} = \frac{1}{0.7} = 1.43$$

主频翻倍，整体只快了 43%，远非翻倍。这证明了主频越大 $\rightarrow$ 计算机越快只是必要条件，而非充分条件。

7 性能评价方法与工具

7.1 CPU 性能评价方法

1. **CPU 执行时间方法**: 直接测量程序在 CPU 上的实际执行时间(前面已详述), 但忽略了 I/O 结构、操作系统等因素对系统性能的影响。
2. **基准程序测试方法**: 利用能反映计算机典型运行情况的一组标准化程序进行测试, 结果用于评价不同计算机的相对性能。

7.2 常用基准程序

基准程序	测试目标	适用场景
Dhrystone	整型运算性能	通用 CPU 整型性能测试
Linpack	浮点运算性能	工作站浮点性能测试
Whetstone	浮点运算性能	微型计算机浮点性能测试
SPEC (含 SPEC-ML)	全面机器性能	全面反映机器性能, AI 算力评测标准
TPC	事务/数据库处理	评测计算机事务处理、数据库处理、企业管理与决策支持等性能

7.3 性能测试工具

- **CPU-Z**: 支持多种处理器和主板型号检测, 提供详细硬件信息。免费。<https://www.cpuid.com/softwares/cpu-z.html>
- **Prime95**: 公认最残酷的烤机软件, 通过烤机测试评估 CPU 稳定性和性能。免费。<https://sourceforge.net/projects/prime95/>
- **PerfDog**: 腾讯开发的跨平台性能测试工具, 覆盖 CPU、内存、GPU、帧率、电量消耗、网络流量等维度。收费。<https://perfdog.qq.com/>

8 【重点】 考试要点速查表

知识点	重要性	核心内容
计算发展历程	【了解】	四时代（手工/机械/电子/量子）；中国四代计算机；ENIAC（1946）
冯·诺依曼结构	【重点】	五大部件；存储程序概念；二进制；冯·诺依曼 vs 哈佛结构对比
ARM 体系	【了解】	ARM V9/V10 特性；SoC 组成；ARM 公司发展里程碑
指令执行过程	【了解】	五步骤：取指令 → 取操作数 → 译码 → 执行 → PC+1
性能指标	【重点】	CPI、MIPS、执行时间、MTBF
主频与速度	【重点】	必要条件非充分条件；受 CPI、存储、I/O 等制约
Amdahl 定律	【重点】	$S = \frac{1}{(1-f)+f/n}$ ；极限加速 $S_{\max} = \frac{1}{1-f}$
性能评价	【了解】	基准程序（Dhrystone/Linpack/Whetstone/SPEC/TPC）；测试工具

8.1 核心公式汇总

必背公式清单

$$CPI = \frac{\text{总时钟周期数}}{\text{总指令条数}}$$

$$MIPS = \frac{\text{指令条数}}{\text{执行时间}(s) \times 10^6} = \frac{f}{CPI \times 10^6}$$

$$T_{\text{执行}} = \frac{\text{指令数} \times CPI}{f}$$

$$MTBF = \frac{\text{系统有效使用时间}}{\text{故障次数}}$$

$$\text{可用性} = \frac{\text{系统有效使用时间}}{\text{系统有效使用时间} + \text{故障修复时间}}$$

$$\text{Amdahl 加速比: } S = \frac{1}{(1-f) + \frac{f}{n}}$$

$$\text{极限加速比: } S_{\max} = \frac{1}{1-f} \quad (n \rightarrow \infty)$$

——祝考试顺利——